

Agentic Business Process Management: A Research Manifesto

Diego Calvanese^a, Angelo Casciani^g, Giuseppe De Giacomo^b, Marlon Dumas^c,
Fabiana Fournier^d, Timotheus Kampik^{e,f}, Emanuele La Malfa^b, Lior Limonad^d,
Andrea Marrella^g, Andreas Metzger^h, Marco Montali^a, Daniel Amyotⁱ, Peter Fettke^{j,k},
Artem Polyvyanyy^l, Stefanie Rinderle-Ma^m, Sebastian Sardiñaⁿ, Niek Tax^o, Barbara
Weber^p

^aFree University of Bozen-Bolzano, Bozen-Bolzano, Italy

^bUniversity of Oxford, Oxford, United Kingdom

^cUniversity of Tartu, Tartu, Estonia

^dIBM Research, Haifa, Israel

^eUmeå University, Umeå, Sweden

^fSAP, Berlin, Germany

^gSapienza Università di Roma, Rome, Italy

^hpaluno (Ruhr Institute for Software Technology), University of Duisburg Essen, Essen, Germany

ⁱUniversity of Ottawa, Ottawa, Canada

^jGerman Research Center for Artificial Intelligence (DFKI), 66123 Saarbrücken, Germany

^kSaarland University, 66123 Saarbrücken, Germany

^lThe University of Melbourne, Parkville, VIC 3010, Australia

^mTUM School of Computation, Information, and Technology, TU Munich, Garching, Germany

ⁿRMIT University, Melbourne, Australia

^oMeta, London, United Kingdom

^pUniversity of St. Gallen, St. Gallen, Switzerland

Abstract

This paper presents a manifesto that articulates the conceptual foundations of *Agentic Business Process Management* (APM), an extension of Business Process Management (BPM) for governing autonomous agents executing processes in organizations. From a management perspective, APM represents a paradigm shift from the traditional view on business processes. This shift is driven by the realization of process awareness by agent-oriented abstractions: software and human agents act as primary functional entities that perceive, reason, and act within explicit process frames. Thus,

Email addresses: diego.calvanese@unibz.it (Diego Calvanese),
casciani@diag.uniroma1.it (Angelo Casciani), giuseppe.degiacomo@cs.ox.ac.uk
(Giuseppe De Giacomo), marlon.dumas@ut.ee (Marlon Dumas), fabiana@il.ibm.com (Fabiana
Fournier), tkampik@cs.umu.se (Timotheus Kampik), emanuele.lamalfa@cs.ox.ac.uk
(Emanuele La Malfa), liorli@il.ibm.com (Lior Limonad), marrella@diag.uniroma1.it
(Andrea Marrella), andreas.metzger@paluno.uni-due.de (Andreas Metzger),
marco.montali@unibz.it (Marco Montali), damyot@uottawa.ca (Daniel Amyot),
peter.fettke@dfki.de (Peter Fettke), artem.polyvyanyy@unimelb.edu.au
(Artem Polyvyanyy), stefanie.rinderle-ma@tum.de (Stefanie Rinderle-Ma),
sebastian.sardina@rmit.edu.au (Sebastian Sardiña), niek@meta.com (Niek Tax),
barbara.weber@unisg.ch (Barbara Weber)

APM moves away from automation-oriented BPM toward systems in which autonomy is constrained, aligned, and made operational through *process aware* agents.

We introduce the core abstractions and architectural elements required to realize APM systems and elaborate on four key capabilities that agents in APM systems must support: *framed autonomy*, *explainability*, *conversational actionability*, and *self-modification*. These capabilities jointly ensure that agents’ goals are aligned with organizational goals and that agents behave in a framed yet proactive manner in pursuing those goals. We discuss the extent to which the capabilities can be realized and identify research challenges whose resolution requires further advances in BPM, AI, and multi-agent systems. The manifesto thus serves as a roadmap for bridging these communities and for guiding the development of APM systems in practice.

Keywords: business process management, autonomous agents, agentic AI, framed autonomy, explainability, conversational actionability, self-modification

1. Introduction and Motivation

With the advent of big data analytics, machine learning, and foundation models—notably, Large Language Models (LLMs)—organizations increasingly adopt data-driven artificial intelligence (AI) approaches to realize software systems that form the backbone of their operations. A long-term ambition leveraging these technologies to instill *autonomy* into software systems, meaning that they independently perceive, reason, and act to achieve their goals, emphasizing intentionality, goal-directed behavior, and constrained autonomy [1]. This lifts the burden of low-level decisions from knowledge workers and can ultimately reduce costs, alleviate workforce shortages, foster creativity and competitiveness, and improve the experiences of employees, customers, and other stakeholders involved [2]. Currently, efforts to achieve greater autonomy in software systems often attempt to make use of LLMs and are summarized under the umbrella term of *agentic AI*, thus promoting the notion of *agents* and *Multi-Agent Systems* (MAS) as the fundamental abstractions of software systems [3].

However, increasing the autonomy of LLM-based software agents also brings risks. On the business level, such agents may make decisions that are in violation of compliance rules, disregard social norms and expectations, or simply lead to directly adverse business outcomes [4]. On the technical level, reliance on LLMs entails inherent functional uncertainty that is difficult to test and debug, leading to system integration and maintenance challenges and, ultimately, to technical debt when replacing software agents requires making sense of entangled behaviors emerging from agent interactions.

To mitigate and control these risks, it is crucial to *govern* software agents in organizations. Agent governance has been a well-established line of research for several decades [5]. It originated primarily in the AI subfield of *normative MAS*, which focuses on regulating autonomous agent behavior through the specification of norms such as obligations, permissions, and prohibitions to ensure social order and compliance [6]. On the more practical side, LLM providers have issued recommendations for governing LLM-based agents [7]. Still, long-running research lines and pragmatic practice-based

guidelines for governing agents lack a bridge to a practically well-established perspective of managing work in organizations.

Such a perspective is provided by Business Process Management (BPM) literature and practice. BPM is concerned with the management of (sets of) activities that are performed in coordination and jointly contribute to the accomplishment of organizational (business) goals [8, 9]. With the advent of AI agents in modern organizations and the delegation of some activities to autonomous agents, it is imperative that BPM also encompasses the management of such agents acting within organizations. Indeed, BPM and MAS research have a strong joint tradition, dating back to the 1990s (cf. [10] for an overview). Still, most of the corresponding work focuses on allocating agents to execute tasks within a business process, rather than on managing autonomous agents as first-class citizens to ensure the achievement of business goals. While some exceptions exist—notably, work on process choreographies [11] and the emerging research directions of *agent system mining* [12, 13], as well as of *agentic AI process observability* [14], within process mining—there is a lack of a holistic perspective on what is needed to apply and extend BPM to the governance of software agents in organizations. Moreover, BPM practitioners often lack a clear understanding of what constitutes an agent (and its associated benefits and risks), resulting in overlooked insights and missed opportunities for meaningful progress in the emerging domain of *Agentic (Business) Process Management* (APM) [10].

Consider an example of an APM system that facilitates the onboarding of new suppliers as part of a procurement process [15]. In such a system, there is a buyer agent and several supplier agents, some of which are human and some AI-based. Both types of agents (buyer and supplier) possess process awareness in the sense that they continuously align their individual goals and act in concert to meet the organizational objectives of the procurement process.

To address the above lack of a holistic perspective, this manifesto introduces the APM paradigm, with the notion of *framed agency* at its core. Specifically, we provide a joint position of academics focusing on both theoretical and applied aspects of the fields of BPM and autonomous agents, as well as that of industry experts. We identified and discussed the required capabilities of APM and associated research challenges during the Dagstuhl Seminar #25192 (AUTOBIZ¹) and follow-up sessions. Specifically, after a series of talks by experts, participants split into working groups to further discuss individual topics of the research agenda. The results of these breakout groups were presented to all seminar participants, and their feedback was used to improve the findings. These were further discussed during the PMAI’25 workshop², culminating in the vision of APM systems.

We first present the underlying concepts and core definitions of an APM system, followed by the main architectural elements, highlighting the key distinctions between the APM system level and the agent level (Section 2). Then, we discuss four key

¹See <https://www.dagstuhl.de/25192>. We express our gratitude to the Scientific Directorate and staff of Schloss Dagstuhl for their invaluable support. We also thank our fellow participants for their engaging discussions.

²<https://ceur-ws.org/Vol-4087/>; intermediate proposals that this paper extends are provided in [16, 15, 17, 18].

capabilities that are required to achieve responsible and effective management of AI agents in business processes: *framed autonomy*, *explainability*, *conversational actionability*, and *self-modification* (Section 3). These capabilities are purposefully ordered: Framing ensures that agents are process-aware and their actions are guard-railed. Explainability serves as a means to preserve the system’s autonomy by having the agents articulate the rationale for their behavior, a prerequisite to deploying them. Conversational actionability then ensures effective execution, interaction, and governance. Finally, self-modification allows for continuous improvement to help move towards the long-term vision of self-improving software systems in the operational back-ends of organizations. The realization and delivery of these four key capabilities involves a series of research challenges, which are presented in Section 4. We conclude the paper with critically reflections and call for the establishment of APM formal foundations, as well as engineering and management practices, and for empirical research to confirm or challenge the assumptions made in this manifesto (Section 5).

2. Agentic Business Process Management Systems

This section first defines the fundamental concepts underlying an APM system and then elaborates on the main architectural elements that constitute such a system at both *macro* (system) and *micro* (agent) levels.

2.1. Fundamental Concepts

In line with the AI and software engineering communities [19, 20, 21], we introduce the concept of an *agentic system* as a collection of (one or more) *individual* goal-driven agents that sense, reason, and act upon external stimuli to deliver (parts of) the functionality of a software system [19, 20, 22]. We use the term *agentic* to indicate that the APM system is agent-centric: agents constitute the primary functional entities responsible for executing business processes and serve as a major organizational backbone. This marks a paradigm shift from previously introduced AI-augmented BPM systems [23] and other *non-agent-centric* BPM approaches, where the term denotes that certain core entities (i.e., not agents) may exhibit some degree of agentic characteristic (e.g., autonomy), see [24].

In an APM system, an *agent* is an autonomous entity situated in an environment [25, 20]. Functionally, an agent operates through a continuous control loop [26], alternating between *sensing* its environment to update its internal state, *reasoning* to select actions that align with its goals, and *acting* to influence the environment. An agent is *proactive* and has its own thread of control, meaning that it makes (semi) autonomous decisions about which actions to perform and when [10, 27]. In this sense, an agent processes information, generates content and knowledge, and proactively makes decisions and performs activities and tasks. This positions agents in contrast to classical components, objects, and services, which do not have goals and respond reactively to user prompts or inputs provided via an API [20, 28, 29]. For example, an agent may browse Web pages and make online purchases on behalf of a user; it may compare prices, select items, and complete checkouts [30].

The notion of an agent is sufficiently general to design environments in which heterogeneous entities perceive, reason, and act to achieve designated objectives. Within an agentic system, we distinguish the following categories of agents:

Human agents, such as process workers or process managers.

Software agents [20], which inhabit a software environment and whose tasks are executed by means of program code, *without deliberation*.

(Physically) Embodied agents [31] (a.k.a robots), whose decision-making and action directly affect the physical world.

AI agents, that accomplish tasks by means of deliberation via AI algorithms and models, for example, using generative AI techniques such as LLMs [32, 33].

While an agentic system hosts a collection of inter-operating agents (i.e., it is a socio-technical system), it does not by itself realize any shared form of *process awareness*. Here, we define process awareness as the assurance that agents' inner workings conform to organizational processes and adhere to their operational constraints, regulations, and goals. Conceptually, we therefore regard an APM system as a system that explicitly realizes such process awareness in some concrete manner.

Accordingly, in the context of BPM, we define an APM system as follows:

Definition 2.1. An *Agentic Business Process Management* (APM) system is a socio-technical system jointly realized by a collection of agents, some of which are at least partially **process-aware**.

An APM system is considered socio-technical in the sense that it allows for a combination of social (human) and technical (technological) agents that interact and depend on each other to function effectively. This does not exclude fully autonomous systems with little to no intervention by any human agent. Collectively, agents execute business processes by virtue of each agent possessing a certain degree of process awareness. Thus, in an APM system, every agent is treated as *autonomous*, with its own goals, decision-making capabilities, and knowledge of (parts of) the business processes. Agents may employ *tools*, owned or shared, that reflect the objects, resources, functions, and services required to fulfill their goals.

We distinguish *autonomy* from *automation*. While automation denotes the execution of predefined tasks or workflows exactly as specified, APM systems facilitate autonomy, whereby agents can perceive, reason, and choose how to act within a process frame in order to achieve given goals. In other words, automation follows fixed rules, whereas autonomy in APM allows agents to make context-sensitive decisions while still respecting process awareness and constraints.

Revisiting our initial APM system example of the supplier onboarding process (Section 1), we can see there are two types of process-aware agents: buyers and suppliers, some human and some AI-based. The buyer agent is responsible for periodically disseminating Requests for Quotations (RFQs) to suppliers identified as suitable for meeting the manufacturing process targets and, in turn, evaluating the quotes they provide in order to decide which supplier to contract with. The supplier agents work to

fulfill their winning bids within individual time frames determined by the contractual terms. Each individual agent may also pursue other, non-process-aware goals that reflect the agent’s specific role. For example, a supplier agent may be restricted to a particular geographical region. An APM system may also include agents that are not process-aware, such as a designated legal agent responsible for ensuring the diligence of the contracts.

We remain agnostic in this example about the specific mechanism used to instrument the agents with process awareness (e.g., how their individual knowledge and goals are set and updated), as there may be different ways of framing it within an APM system. For the purposes of defining the essence of an APM system, however, it is important that APM agents incorporate some concrete realization of such a framing capability (see Section 3.1).

Considering the agent as a first-class citizen in an APM system, we define an agent as follows:

Definition 2.2. An *agent* in an APM system is a primary execution entity—an actor that perceives, reasons, and acts autonomously, with its autonomy framed to ensure process-aware behavior aimed at achieving process goals.

We may call an agent in an APM system an *APM agent*. It is important to note that once an agent becomes part of an APM system, its individual behavior and goals (i.e., micro-level work execution) are both framed and aligned. The former implies that the agent’s behavior is directed at both complying with process constraints (the *frame*) and towards achieving the goals of the APM system processes (i.e., the macro level). The latter entails that its behavior is harmonized with that of the other agents in the system. While being process-aware is an inherent property of an APM agent, it is a qualitative property in the sense that different agents in an APM system may vary in their degree of process awareness and autonomy, and in the extent to which their behavior helps achieve the system’s collective goals.

To realize this notion and equip an agent with the means to promote autonomy and process awareness in an APM system, we identify four essential capabilities the agent must possess: framing, explainability, conversational actionability, and self-modification. These capabilities, and the rationale for their inclusion, are further detailed in Section 3.

The actions of an agent are facilitated by the use of tools. We therefore define:

Definition 2.3. A *tool* in an APM system is a means accessible to an agent that augments its capacity to reason, and to perceive and act upon its environment.

Possible tools may include sensors, actuators, messaging, or other software functions and services. These tools may be accessed through communication protocols that define how information is exchanged.

In light of the presented concepts, next, we elaborate on the building blocks of an APM system and of an agent.

2.2. Conceptual Architecture

This section introduces the conceptual architecture of an APM system. As visualized in Figure 1, the architecture focuses on two distinct levels: the *APM system* rep-

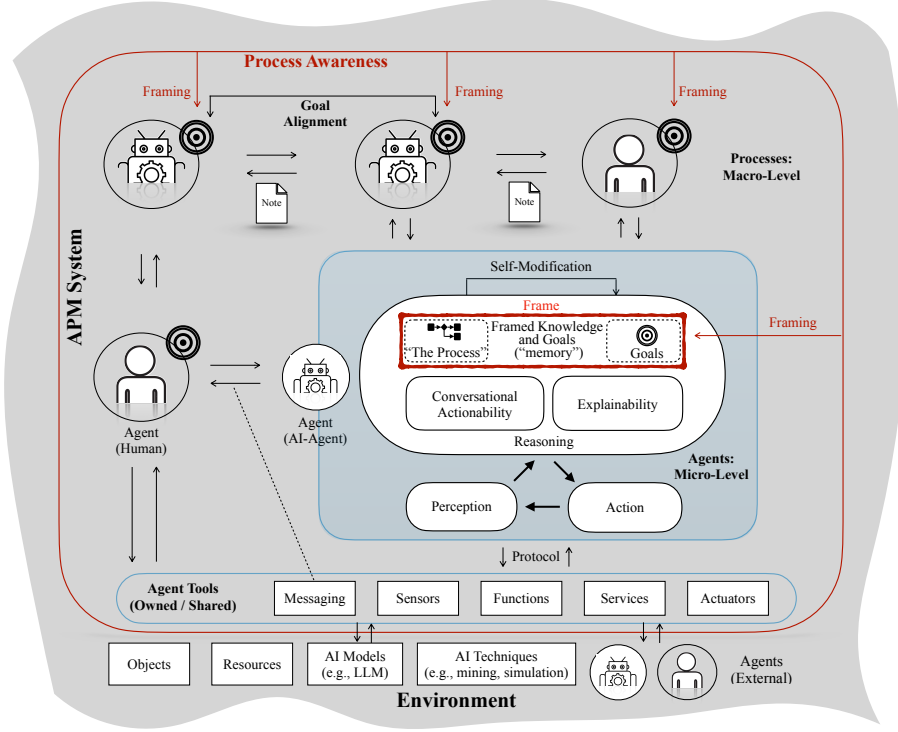


Figure 1: Conceptual architecture of an APM system depicting its key actors, components (boxes), and interactions (arrows). *Process-awareness* (red-colored) is realized by a framing mechanism that constrains and aligns agents at the macro level, while at the micro level each process-aware agent runs a *Perceive–Reason–Act* loop over framed knowledge and goals (its internal view of “the process” and its goals), as agents interact with other process-aware human- and AI-agents in the system, and with the external environment (external agents, objects, resources, AI models/techniques), using tools (messaging, sensors, services, and actuators).

representing the *macro level* management of work that establishes the process-awareness of its agents (Subsection 2.2.1), and the *APM agents*, i.e., the autonomous execution of work in the APM system on the *micro level* (Subsection 2.2.2).

2.2.1. APM System

The APM system constitutes the “world” where agents live and act, and in which they can interact with each other and with the environment via tools (both individually owned and shared). These tools let the agents exchange data (e.g., *notes*, *messages*) with other agents in the APM system, employ a variety of AI models (e.g., LLMs) and techniques (e.g., simulation, mining), sense and manipulate external objects and resources, invoke remote functionalities via services, and interact with external agents that are not part of the APM system. To understand each other, agents rely on a shared language, ontology, or protocol [20]. Examples include Model Context Protocol (MCP), Agent Communication Protocol (ACP), Resource Description Framework

(RDF), and Knowledge Query and Manipulation Language (KQML).

The *macro level* (process-aware management) represents the *processes* themselves, defined by collective, organization-level goals. The attainment of these goals is achieved by imposing process awareness and goal alignment on the agents through a *framing* mechanism.

Definition 2.4. *Framing* is a primary mechanism for ensuring process-awareness and goal alignment in an APM system, imposing restrictions on the autonomy of agents through their knowledge and goals.

Without framed autonomy, an agent could perform any action in pursuit of its own individual goals. It provides the normative and operational specifications that agents must adhere to, including the *lifecycle management* to create, suspend, or destroy agents as the process evolves. More concretely, the realization of the framing mechanism attends to two aspects:

Process-Awareness governs all agents' work toward common process objectives. Process awareness entails that an agent's autonomy is directed to ensure that its actions aim to fulfill the collective process goals.

Goal-Alignment sets the rules of engagement and structural relationships (e.g., hierarchies or coalitions), including the coordination of goals among agents that may work collaboratively. This also reflects the assignment of roles and segregation of duties throughout their lifecycle.

The functional manifestation of the framing mechanism is expressed through the capabilities of the agents (see next section), while its concrete realization is an engineering choice, since there are various ways in which it can be implemented—for example, by assigning collective responsibility to one or more agents, through orchestration, or via a shared memory (e.g., as part of the frame) that allows agents to progress along a mutual plan.

2.2.2. APM Agent

The *micro level* (agent-oriented execution) consists of a collection of autonomous agents that jointly execute the process—and may participate in multiple processes.

Drawing on foundational ontologies like DOLCE [34] and GFO [35], agent-oriented modeling frameworks like *i** [36] and Tropos [37], and classic AI constructs such as BDI [38] and FIPA [39], as well as the mental model theory of reasoning [40], and in concert with the characteristics of an AI-Augmented BPM system [23], we consolidate here a unified specification schema for an APM agent.

As conceptualized in Figure 1, an agent, from an architectural point of view, consists of three main conceptual modules [20]:

The **perception module** allows the agent to observe the state of its *environment*, *context* (situational constraints, location), and other agents. These agents operate within the agent's environment: some are part of the APM system (and thus process-aware), while others may be external to it. This module is responsible for managing the agent's *perception and sensing* attributes (e.g., sensors, percepts, belief update functions), ensuring the agent continuously updates its understanding of the world.

The **reasoning module** is the core component for knowledge representation and decision making. It processes sensory data and is continuously informed by the framing mechanism to maintain the agent’s *mental* (i.e., knowledge, perceptions, and beliefs) and *intentional* (i.e., desires, intentions, and goals) models. This also includes the agent’s ability to self-modify, learn, and explain its reasoning. The agent’s core knowledge includes its *identity* (i.e., agent ID, type), along with the agent’s *behavioral dispositions and plans* (policies, triggers, reactions) to decide what to do next. This module also inherently provides a computational mechanism for decision-making, enabling the agent to continuously process its knowledge, revise and generate ongoing inferences, and translate its perceptions into actions. Recent advances in AI explore the use of LLMs for this purpose, although their true capability to fulfill this role remains under debate [41, 42]. Several alternative approaches could support such a competence within an agent, including externalizing parts of the reasoning through specialized tools. Similar to the framing functionality, we intentionally leave the concrete realization of the decision-making component open.

The conceptual architecture introduces the following reasoning-related capabilities required for a generic agent to become *process-aware* within an APM system:

Framing (see 3.1). This module represents the internalization of the APM system’s *framing*. Uniquely for an agent in an APM system, as a process-aware actor, the agent’s knowledge includes two key internal elements: a mental model and an intentional model. The internal mental model serves to represent the process reality, viewed by the agent as *the process*, as perceived through the framing mechanism. It serves as the agent’s “memory” regarding the process model, running instances, stakeholders, and all relevant process context. The internal intentional model maintains the agent’s goals as they are shaped by its *social* and *normative* dispositions (i.e., obligations, roles, collective goals, and prohibitions). By explicitly consulting these frames, the reasoning component ensures that the agent’s local constraints and objectives remain aligned with those of the overall process.

Explainability (see 3.2). Supported by the agent’s *accountability* and *traceability* attributes (i.e., trace logs, auditability), *explainability* is the ability to articulate the rationale behind decisions. This ensures that process stakeholders can receive specific explanations of the workings of the socio-technical system.

Conversational Actionability (see 3.3). Building on the agent’s *communication* and *interaction* attributes (i.e., protocols, roles, languages), this capability allows the agent to negotiate and coordinate with other agents and act on the environment. It also enables process stakeholders to trigger actions, either at design- or run-time, to change system behavior.

Self-Modification (see 3.4). Grounded in the agent’s *perception* abilities, self-modification is an agent’s capability to adapt and evolve over time. Agents self-modify to better reach individual goals and, when successfully *framed*, to achieve collective process goals.

Finally, the **action module** allows the agent to perform actions that change the state of

the environment and send messages to other agents. It defines the agent’s *capabilities* (i.e., skills, resources) and abilities to socially interact with other agents.

3. Envisioned Capabilities of an APM system

This section gives an overview of the aforementioned key capabilities of an APM system: *framing*, *explainability*, *conversational actionability*, and *self-modification*, as initially proposed in [16, 15, 17, 18].

3.1. Framing in an APM system

The guard-railing of an autonomous agent requires assuring it is operating within its current *frame*. Intuitively, a frame is a set of rules, restrictions, and regulations, which may evolve over time. Frames establish boundaries within which one or several agents in an APM system may operate with maximal flexibility, making autonomous decisions [43]. Frames may exist—at least—on *agent type*, *process*, and *organization* levels (as well as potentially across organizations).

More analytically, frames are *normative*: they specify deontic process requirements governing the behavior of the process. In contrast, classical process specification languages, such as BPMN and DECLARE, are primarily concerned with specifying behavior required to accomplish a business goal³. However, unlike informal definitions of business processes, e.g., as “sets of activities” performed to “jointly realize a business goal” [8, p. 5], goals remain largely implicit in these more formal and operationalizable process specification languages.

In previous works, operational specifications have been called *frames* as well [23]. Indeed, they can be considered a sort of *operational* frame. In APM systems “frames” focus, however, on the normative specification. When we need to distinguish, we call the two frames *normative frame* and *operational frame*, respectively. For example, an agent may be specified so that it must reject any student assignment submitted after the deadline. Such a requirement is considered a form of normative frame statement. An operational frame specification may complement this statement by instructing that, when rejecting an assignment, the agent shall: (1) retrieve the name and ID of the student who created the assignment, (2) record these details in the database, and (3) send the student an email informing them about the rejection. Alternatively, these three operational statements could be replaced by a single prohibition statement (i.e., a normative frame) not to delete the rejected assignment without informing the student who submitted it, leaving more room for autonomy in the behavior pursued by the agent, for example letting the agent decide on its own whether to first send an email to the student and then record the information in a database, or the other way around.

If there are no autonomous decision-makers, then the normative frame is just an additional condition over the operational frame; but if decision-making is possible,

³BPMN is *imperative*, specifying—at least ostensibly—the exact control flow of activities to be executed, and is therefore well-suited for *operational* requirements. DECLARE, in contrast, is *declarative*, specifying constraints over permissible behavior. While this declarative nature aligns naturally with *normative* requirements, DECLARE is typically used to constrain execution rather than to explicitly capture deontic notions such as obligations, prohibitions, or permissions.

then the operational frame requires finding a strategy to satisfy the objective, whereas the normative frame requires choosing a strategy that remains within what is allowed (with respect to the frame).

Strategies for achieving goals under framed autonomy are associated with decision-makers, including software agents, giving rise to several problem setups, for centralized as well as distributed intelligence.

Centralized intelligence: We consider the “AI agents” as a single entity orchestrating the *process* that is executed in a mutually fully observable and coordinated manner. The *environment* may be stochastic and not fully observable. The frame is over the process. The single entity may have active or passive responsibility for the frame. If we have multiple agents, we may break down the problem into several of the above scenarios.

Distributed intelligence: We consider AI agents as distributed entities that enact the process as *resources*. This has wide-ranging implications: a resource may have only partial observability of what other resources are doing; coordination may be effortful, and resource-level goals may be mutually inconsistent, or inconsistent with process-level goals. In such scenarios, we can frame individual resources, groups of resources, or the entire process. Accordingly, we need to assign responsibility to individual agents or groups thereof, and there may be strategic interactions affecting responsibility.

From these problem setups, we can derive three different blueprint scenarios for framed autonomy in business processes:

- (i) we have a single decision-maker and place a frame on process behavior;
- (ii) we have multiple decision-makers and place frames on individual decision-makers;
- (iii) we have multiple decision-makers and place frame(s) on process behavior or parts thereof.

In practice, there may be additional variance to the scenarios. For example, normative frames may be partially represented within operational process specifications, restricting overall agent autonomy. An example is a purchasing process where purchase orders can only be created and paid through a central IT system that enforces normative rules, e.g., regarding four-eyes approval policies. Other parts of the global normative frame can potentially be projected to local agent-level norms. For example, overall spending limits may apply on the global level, but could be operationalized locally. Specifically, local operationalization of frames acts as a security and privacy boundary. Indeed, the frame limits an agent’s perspective and capabilities to only its necessary process context, inherently restricting the potential impact if an agent is compromised or manipulated by a deliberately malicious actor.

3.2. Explainability in an APM system

As a prerequisite to effective execution and governance, an APM system should be explainable. Because an APM system is a composition of AI agents, this may be achieved by endowing agents with the intrinsic capability to explain their own behavior

and actions, including those agents assigned collective process-awareness responsibilities. To this end, each agent’s specification should include explicit instructions concerning the requirement to articulate the explanans (the explanation itself) in response to certain triggering situations or conditions (explanandum), as well as which explanation mechanisms—so-called *eXplainable AI* (XAI) techniques—may be employed.

Empowering agents in APM systems with explainability will help address important concerns, including the following:

Trust issues may arise among stakeholders—including process owners, business analysts, end users, and customers—who may hesitate to rely on agentic process recommendations or automated agent decisions when the underlying rationale is unclear.

Accountability for agent behavior requires that if an agent fails, it can explain the underlying rationale such that responsibility can be assigned and corrective actions can be implemented.

Biases may be perpetuated by AI and ML components underlying APM systems and their agents. Such biases may lead to discriminatory or unfair agent decisions; explainability is a prerequisite for detection and mitigation.

Compliance of agents’ behavior with regulatory frameworks, such as the EU’s GDPR and AI Act, needs to be demonstrable. This requires an increasing level of transparency, particularly in high-risk domains like finance, healthcare, and human resources, which are common areas for BPM applications.

Agents in APM systems make independent decisions, adapt to changing conditions, and learn from experience with minimal human intervention. Here, explainability offers a central mechanism through which agents can articulate the rationale behind their behavior. As such, explainability becomes a first-class citizen in the realization of APM systems, supporting agent autonomy from two perspectives: (i) enabling agents to independently resolve misalignment in other agents’ behavior; (ii) reducing human intervention by making agent behavior understandable and transparent.

To account for the quality of explanations in APM systems (e.g., soundness, accuracy, usefulness, and interpretability), derived from situation-aware explainability [44, 45, 46] and causal processes [47, 48], we postulate that the following are desirable properties about explainability in APM systems:

1. Ability to infer explanations that conform to the framing constraints and statements.
2. Ability to capture the richness of contextual information that affects agents’ behavior and decisions.
3. Ability to reflect causal execution dependencies among actions in the agents’ trajectories/behavior.
4. Ability to provide explanations that are interpretable to other agents (humans or digital).

3.3. Conversational Actionability in an APM System

As the term suggests, conversational actionability refers to an agent's ability to combine interaction and enactment capabilities. This is essential in an APM system. On the one hand, agents need to integrate conversational capabilities to coordinate with one another and to receive instructions from, interact with, and report to the human agents involved in 'the process' in various roles (ranging from active participation to management). Knowing what the process is and who the agents assigned to relevant roles are may be considered part of an agent's mental model of the process. On the other hand, agents must be able to link such conversations to the corresponding enactment capabilities, making decisions and performing actions accordingly.

In [17], conversational actionability is positioned within AI-augmented BPM Systems as the ability of the system to handle these two requirements:

Process-aware conversation. The system can interact with users or external agents using a conversational interface to support, trigger, and guide actions related to the enactment of one or multiple processes.

Process-aware actionability. Conversations trigger business process executions, such as taking a decision or performing an action, as well as evolution actions, such as deciding on priorities or changes to the process.

When moving from a centralized execution system to an APM system, these capabilities must be collectively realized by the set of agents operating within the system. Instead of relying on centralized intelligence and a single point of truth, an APM system caters to federated perspectives. Each agent decides how it instantiates these two capabilities based on its own individually informed perspective about 'the process' (i.e., its frame). Hence, this may require the use of consensus resolution approaches or the ability of external agents to tolerate a possible variety among the results when interacting with different agents.

As for the conversation component, an agent may, in fact, interact with other agents using natural language and/or exploiting other unstructured and semi-structured information sources (such as diagrams and charts) or formal communication languages, such as multi-agent interaction protocols. Choosing which interaction modalities are embodied by the agent, manifested by the proper selection of tools in the APM system, obviously also depends on whether an agent only interacts with other computational agents, and/or humans.

In terms of actionability, depending on the role played by the agent, it may need to cover one or more key functionalities (i.e., types of agent-enacted behavior; cf. [17]):

Query – to provide information on the process(es), either at the model level or regarding execution data pertaining to the current or past states of affairs;

Recommend – to provide insights and suggestions on the adaptation and future evolution of process instances;

Create – to elicit models from domain knowledge and process-relevant data;

Execute – triggering actions to move process instances to a new state.

In order to enact such behaviors, agents typically require interacting with tools providing different services or the same service with different functional and non-functional

guarantees. For example, determining the likely time-to-completion for an order may be achieved using predictive monitoring techniques, employing simulation, or by a combination of the two. This creates the challenge of identifying the best mix of tools to realize an overall functionality in the “best way” possible (see Section 4.3).

3.4. *Self-Modification in an APM System*

In order to adjust to ephemeral or permanent changes, APM systems must be able to self-modify. A fundamental distinction in self-modifying APM systems is between *adaptation* and *evolution* of both individual agents and agent coalitions. Adaptation and evolution fundamentally differ in scope, duration, permanence, and the type of knowledge they leverage.

Adaptation refers to short-term, instance-specific modifications [49, 50] that address immediate, unforeseen issues during process execution. These modifications do not alter the underlying process model or schema, but rather constitute localized modifications performed by the agent within the constraints of the current process definition to handle exceptional circumstances or environmental changes that were not anticipated at design time. Adaptations are ephemeral—they affect only the current process instance and do not propagate to future executions unless explicitly learned and incorporated into the model (which then would be considered evolution). Such adaptations can be enacted individually by an agent, or they can be enacted collaboratively by engaging with other agents about how their shared perspective on the process may need to be adapted. Adaptations may be reactive (i.e., in response to perceived deviations or problems) or they may be proactive (i.e., performed based on predictions), see [18, 51].

Evolution describes long-term modifications to the process logic, model, or policy that persist across multiple instances and executions. These longer-term modifications are typically shared among multiple agents in the APM system. Evolution may be informed by aggregated insights, patterns, and correlations discovered through analysis and learning from historical execution data, performance metrics, and repeated anomalies. Rather than responding to a single event, evolution synthesizes knowledge from multiple instances to identify systemic issues, improvement opportunities, or changing environmental conditions that warrant permanent changes to how the process is defined or executed. These modifications affect the process model itself and thereby influence its future instantiations, i.e., future processes [52]. Such evolution also entails the need to communicate and re-align how it is applied across all relevant agents in the system.

Ideally, the relationship between adaptation and evolution forms a feedback cycle in the APM system. Adaptations generate execution traces and performance data that feed into agents’ learning mechanisms. When certain adaptive patterns prove consistently effective across multiple instances or contexts, they become candidates for evolutionary incorporation into the process model [53], and hence into the agent’s frame. Conversely, evolution should reduce the frequency of certain types of adaptations by preemptively addressing known failure modes or inefficiencies. However, adaptations remain necessary for handling truly novel situations that have not been anticipated during design time [54] or not yet encountered frequently enough by any agent in the APM

system to justify evolutionary modifications, or to address context-specific conditions that should not be generalized.

The distinction between adaptation and evolution has important implications for system design. Adaptation mechanisms must prioritize responsiveness, robustness, and security/safety under uncertainty, often operating with incomplete information and limited time for deliberation. Adaptation mechanisms require runtime monitoring, exception handling capabilities, and flexible execution engines that allow deviating from prescribed process execution paths. Evolution mechanisms, meanwhile, require sophisticated analytical capabilities: pattern mining across execution histories, causal inference to distinguish correlation from causation, statistical validation to ensure that observed patterns are not artifacts of noise or bias, and change management protocols to securely/safely deploy model modifications without disrupting ongoing operations.

4. Challenges

This section provides an overview of research challenges that require solving to further advance the relevance of APM. The challenges are categorized according to the four core capabilities of *framed autonomy*, *explainability*, *conversational actionability*, and *self-modification*. Analogously to the capabilities in the previous section, these challenges were identified during the Dagstuhl seminar and published as PMAI workshop papers at ECAI [16, 15, 17, 18]. We augmented and refined these challenges based on the instrumental feedback received from the PMAI workshop participants.

In addition, we introduce a set of *cross-cutting* challenges that affect several of the aforementioned capabilities.

4.1. Challenges Regarding Framed Autonomy

Achieving framed autonomy in APM systems comes with practical challenges, ranging from fundamental questions about the notion of an agent in business processes to specification and operationalization.

F1: What is a pragmatic notion of an agent in the context of business process execution? Before the broad adoption of LLMs, the notion of an agent did not play a major role in the engineering of business information systems and the processes that run them. Consequently, practitioners cannot be expected to be familiar with the depth and sophistication of agent-related abstractions and the presented framing mechanism for the realization of process awareness. To the contrary, a practitioner may consider a software tool that makes use of an LLM as an agent, without much thought about further properties. Defining a more precise and robust notion of an agent that is still intuitively understandable by business process management practitioners can thus be considered a key prerequisite for framing agents' autonomy.

F2: How to elicit and specify frames? The elicitation and specification of mental and intentional frames requires a *frame meta-model*, and one or several specification languages. To this end, existing specification languages can be reused; potentially, several languages and their underlying concepts can be combined. For example, declarative

approaches to process specification—such as DECLARE [55] and in more practical contexts business rule and query languages with temporal reasoning capabilities [56]—can be augmented with deontic notions to promote normativity to a first-class abstraction. For elicitation, both symbolic and subsymbolic approaches can be used and fused. LLMs can generate frames or parts thereof from natural language text, whereas rule mining approaches can be applied to infer normative constraints from the traces of well-behaved agents and multi-agent systems.

F3: How to operationalize frames on real-world symbolic data? Once specified, frames need to be integrated with APM systems to ensure the agents’ frame-compliance during runtime. A short- to mid-term prerequisite is the operationalization of frames using technologies that do, in fact, run in large organizations. Here, explainability is a necessity, considering the practical intricacy of normative requirements, as well as the scale of real-world symbolic queries and data. To address this runtime compliance, formal logic-based approaches [57] can provide the imperative logic foundation needed to enforce the operational boundary of the agent. By leveraging strict transition semantics, such formalizations can naturally guardrail the agent’s execution to legal actions and safely handle exceptions via guarded execution constructs.

F4: How to design incentive and reinforcement mechanisms for frame-compliant agents? A central challenge in APM is understanding how to design agents whose reasoning and behavior consistently align with their prescribed frame, meaning the norms, goals, constraints, and process logic meant to guide their actions. The question is not merely how to make such a frame available to agents, but how to ensure that agents actively use it, internalize it, and treat it as instrumentally valuable. This requires developing ways to define, measure, and strengthen an agent’s degree of frame compliance as a continuous, learnable property. A prerequisite are incentive structures that cultivate an agent’s “desire” to be process-aware. Humans can be motivated through monetary compensation, perks, recognition, or intrinsic cultural alignment. AI agents require a functional counterpart that reshapes their internal utility models. The open problem is how to design rewards inherently as part of the frame, that make adherence to the frame beneficial for the agent while still preserving autonomy. To support this, reinforcement mechanisms must be woven directly into the agentic lifecycle. Over time, agents must learn which components of the frame are genuinely useful, which elements constrain them productively, and which parts might be deprioritized without violating safety, governance, or organizational policies.

4.2. Challenges Regarding Explainability

Challenges with respect to the explainability in APM systems center around technically sound explanations that are easily understood and actionable, given their intended and actual explainees.

X1: How to specify or let agents learn other agents’ preferences regarding explanations? A variety of elicitation mechanisms may be used in an APM system to effectively capture explanation preferences through various channels, whether explicitly

declared upfront in the agentic specification, interactively elicited through dialogue, or implicitly inferred from user behavior.

Systems must accommodate both static preferences that remain consistent and those that dynamically adapt to changing contexts, while supporting the natural evolution of preferences as the explainee’s, human or digital, understanding develops. Here, the challenge is to find the sweet spot between keeping explanations up to date and not confusing the explainee (i.e., limiting its autonomy). Finally, we have to think about when and how to provide full versus incremental explanation updates.

X2: How to leverage and extend existing XAI techniques by the agents to generate explanations? Employing state-of-the-art XAI techniques for APM systems has several limitations. This is because making sense of agent behavior goes beyond explaining the AI models used under the hood; it must also account for the broader context in which agents perform their actions, the knowledge they may have acquired over time, possible cause-and-effect relationships among their actions, and the constraints (i.e., framing) imposed on their behavior. We therefore need new and enhanced explainability techniques that help agents explain the outcomes of their behaviors and decisions more broadly.

X3: How may one articulate actionable explanations (e.g., to other agents) to preserve autonomy? Besides being context-sensitive, explanations should be actionable—indicating to the explainee which corrective or mitigating actions could be taken to alter the state of the explanandum, particularly without escalating the situation to any external agent. In this way, the explainee may be able to autonomously act upon the condition at hand. However, further work is needed to devise a systematic approach that enables the explainer to determine the most effective content for the explainee, to elicit such corrective action—taking into account both the explanandum and the behavioral intentions of the explainee.

X4: When to generate explanations (generation time) and how long to preserve them? With respect to APM system performance and memory efficiency, it is important to assess whether explanations should be generated proactively wherever feasible, or whether their generation should be deferred until necessary. A related question concerns when outdated explanations ought to be retired [58].

X5: How to generate causally sound explanations? Not every pair of action executions is causally dependent. To generate sound explanations, it is therefore important to distinguish spuriously correlated actions (e.g., those that are merely temporally sequential) from causally dependent ones (i.e., when one action directly triggers the execution of another) within an agent’s behavior or across multiple agents.

4.3. Challenges Regarding Conversational Actionability

Challenges in conversational actionability pertain to how agents can realize a virtual circle of action and interaction with human principals such as domain experts and other agents in order to enact the process faithfully and efficiently.

A1: How to engage in conversations with human principals and other agents? The first challenge pertains the *conversational* facet of an APM system. This contains two distinct aspects. The first concerns the need for agents to interact with human principals—typically using natural language or domain-specific languages (such as process diagrams, dashboards, and the like). As for natural language in general, there are only few studies focusing on the ability of foundation models to converse about processes, with many open challenges discussed in a dedicated vision paper [59], and some preliminary contributions [60, 61]. This is even more challenging when natural language conversations are mixed with domain-specific knowledge sources.

The second aspect pertains to agent-to-agent interactions and data exchange, which can be carried out using a variety of different means, from specific interaction protocols and recently developed human-readable token encoding formats (e.g., TOON⁴), and formal languages to more flexible forms (e.g., based on higher-level, informal languages).

A2: How to exploit capabilities of process management tools and services? The second challenge pertains to the *actionability* facet of an APM system. Agents need to relate conversations to goals and, in turn, to actions. This imperative raises a twofold challenge.

On the one hand, the actions in question are not under the direct responsibility of the agents, but require them to interact with different *actuators*, such as: (1) ERP, CRM, or other systems of record to perform transactions; (2) collaboration tools, e.g., to trigger notifications to human actors; (3) physical actuators, e.g., IoT devices or robots; (4) process or task automation tools to update the state of a process or to trigger predefined automation scripts; (5) dedicated planners and schedulers to trigger sequences of steps to achieve a desired state.

On the other hand, in deciding which action should be selected, agents typically need to acquire relevant information about the organisation and, in particular, the current state of affairs as part of their individual mental model about the process. This can only be done if the agents can effectively exploit tools and services encapsulating specific, well-understood, and highly trusted functionality related to data and process intelligence, covering all phases of the process lifecycle (from model-driven analysis and simulation to online data querying, process mining components, as well as predictive and prescriptive components). This calls for encapsulating such components as tools into semantically rich descriptions that can be used by agents, describing functional and non-functional requirements and enabling discovery (for example via MCP).

A3: How to compute and use key indicators on the overall behavior? A key challenge related to conversational actionability is how to compute and use key functional and non-functional process indicators obtained when gathering information and taking actions. Such indicators range from standard KPIs related to performance, time, and cost, to a larger set of indicators measuring/estimating trust, usability, uncertainty, flexibility,

⁴<https://github.com/toon-format/spec>

and alike (see [17]). A set of KPIs to assess completeness and correctness of process models when compared to corresponding textual descriptions have been proposed [62]) and vice versa for generated text from process models in [63]. Moreover, for example, an agent may need to ponder the impact of taking an action and, in doing so, may invoke a simulator and/or a black-box predictor. Each of these components would provide different levels of trust, precision, and uncertainty, and depending on how they are used (e.g., selecting one of them or selecting both and then aggregating the obtained outputs), such levels would propagate up in completely different ways. This is essential to obtain end-to-end indicators regarding the overall process execution.

A4: How to balance autonomy, delegation, and control? The presence of multiple, interacting agents, operating using different tools and AI models with different levels of trust, performance, cost, etc, makes it even more challenging to determine how to balance autonomy and human oversight, and how to achieve the best trade-off between delegation (improving performance) and control (mitigating undesired outcomes and retaining human responsibility).

4.4. Challenges Regarding Self-Modification

Below, we present the relevant challenges related to the capability of APM systems to adapt and evolve.

M1: How to govern self-modification? For the agents in an APM system to operate safely and effectively, governance and human oversight must be built into their design. A key question is when to refrain from autonomy and return control to a human agent, e.g., in scenarios where the autonomous agent has insufficient confidence and is at risk of making mistakes. Research in AI planning, runtime monitoring may help establish thresholds or confidence bounds beyond which a process must escalate to human decision-makers. Techniques from the ML literature on prediction with reject option [64] (sometimes called “learning to defer”) can be explored as a solution direction.

Similarly, determining when and how a human agent should validate a proposed process or policy modification requires a framework for explainable adaptation, where the autonomous agents present justifications for their suggested modifications. Solution directions may include methods from the field of explainable AI (XAI) [65] (see 3.2), or justifications may be provided in alternative forms such as simulations of expected outcomes.

Another major issue is aligning the planning and goals of the AI agents in the system with those of its human agents (i.e., process stakeholders) [66]. Multi-objective optimization techniques [67] can assist in balancing trade-offs between performance, cost, compliance, and user satisfaction while maintaining logical constraints defined in the process model. However, optimizing across conflicting objectives remains a challenging problem, especially when human values or other non-quantifiable criteria are involved.

M2: How to evaluate the success or failure of modifications? Quality assurance in fully autonomous scenarios introduces its own challenges. Without humans routinely checking outcomes, the agents in the APM systems must develop internal mechanisms for evaluating whether their adaptations “worked”. Here, ML-based anomaly detection, performance baselining, and causal reasoning can help detect regressions or misbehavior. Formal methods for verification and validation of modifying process logic or agent policies also present a relevant research direction here. Generative AI, particularly LLMs, could also be used for generating justifications or summaries of decisions for human audits or by providing labels for downstream evaluation—an emerging area sometimes called LLM-as-a-judge [68]. Solutions to the quality assurance problem could use human input purely for evaluation, even in scenarios where the execution is fully automated. However, human input is costly, and hence a challenge is to make this cost-efficient with maximal autonomy. Directions to be explored could include active testing [69] or online testing [70].

M3: How to align concurrent evolution and adaptation of linked process executions among process-aware agents? Business processes rarely exist in isolation. In practice, APM agents operate within ecosystems of interconnected process executions, shared resources, and distributed agent coalitions that may themselves be undergoing evolution and adaptation. For example, agents engaged in related processes may undergo similar adaptations and thus may be able to share “lessons learned” from these common adaptations, thereby making the overall modification more effective (e.g., see [71], where this is explored for self-adaptive software systems). A key challenge is detecting and managing these interdependencies in real-time. APM agents must be aware not only of their own mental and intentional states (i.e., processes and goals), but also of the adaptation state of related agents engaged in the same processes. This requires mechanisms for inter-process communication, coordination protocols, and potentially a meta-level orchestration agent that monitors system-wide adaptations. Techniques from distributed systems, such as consensus algorithms, distributed constraint optimization, and multi-agent coordination [72], may provide foundational approaches. However, these must be adapted to the dynamic, semi-autonomous nature of APM systems, where agents might not have complete visibility into other agents’ internal states or future intentions.

M4: How to enable continuous learning and adaptation management? A defining feature of self-modifying agents in APM systems is their ability to learn from experience and improve process behavior over time. This may be achieved utilizing process mining techniques by the agents themselves as a self-reflection mechanism to analyze their own recorded behaviors (a.k.a., agent trajectories). To support this, the agents must continuously record the adaptations they make and assess their impact both locally (per agent) and globally (across agents). Capturing this meta-knowledge creates a feedback loop where successful modifications reinforce future decisions and ineffective ones are pruned. AI planning and reinforcement learning techniques (while balancing exploration and exploitation; e.g., see [50, 73]) are promising approaches to structuring such learning loops, especially when context-aware and enriched by causal

modeling of intervention outcomes.

However, scaling these techniques may pose challenges: Long-running processes or high-volume APM systems face constraints of bounded memory and context. The agents in the system cannot retain or process each event they have ever observed. Hence, the agents must learn to construct and maintain bounded knowledge representations: compressed summaries, predictive state abstractions, or fixed-size windows of relevant execution history. Techniques from stream reasoning, process mining over sliding windows, and transformer-based sequence models may offer solutions. Designing APM agents that know which parts of their history to remember, forget, or query becomes a core challenge for sustainable, real-time, continuous learning.

M5: How to model and measure uncertainty in APM systems? Non-determinism in agentic behavior may arise in APM systems due to (i) stochastic AI techniques and models (such as LLMs); (ii) stochastic and/or drifting contexts; (iii) the difficult-to-anticipate actions of human agents. The agents in APM systems thus must operate under both aleatoric (inherent randomness) and epistemic (lack of knowledge) uncertainty [74]. It is well-known from the literature on uncertainty quantification and active learning that accurate estimates of specifically *epistemic* uncertainty are a prerequisite to learn and adapt policies to new environments in a data-efficient way [74, 75, 76].

Techniques from probabilistic modeling, Bayesian inference, and fuzzy logic are key for representing and reasoning about agentic uncertainty in process executions. In some domains, thresholds or confidence levels may be set by human experts (qualitative), while in others, Bayesian models or statistical metrics (quantitative) provide actionable measures. As an example, reliability estimates for proactive adaptation may be derived from ensembles of prediction models [77]. APM agents must combine these forms of knowledge, learning from past executions when quantitative models are feasible, while falling back on qualitative heuristics when data is sparse or ambiguous. Hybrid methods that integrate fuzzy logic or ensemble learning could further improve uncertainty modeling in domains with imprecise information. Deciding when to defer the decision-making back to a human agent is a classic task where most solutions involve uncertainty quantification [64].

4.5. Cross-cutting and Complementary Challenges

C1: Toward Automated Provisioning and Onboarding of Legacy BPM into APM. A key challenge in this direction lies in provisioning and onboarding legacy process assets into an APM system. Organizations often operate long-standing BPM and workflow systems, implemented across diverse technologies and custom solutions. Many such systems require extensive manual effort for data handling, exception management, and cross-system orchestration, which in turn introduces operational inefficiencies and error risks. Moreover, critical business logic is frequently fragmented across workflow diagrams, scripts, rules, and tacit human practices, making structured extraction and reinterpretation difficult. Transitioning to APM therefore demands not only technical migration but also semantic and operational transformation, namely *agent-centric process mining*, including the reinterpretation of workflows into agent-oriented roles, goals, policies, and behaviors, while preserving institutional knowledge and ensuring

continuity of governance and auditability. How this process can be aided, automated, deployed, and validated may keep the industry busy for years to come, as organizations seek reliable methodologies and tooling to safely and incrementally modernize their automation capabilities.

C2: Holistic Security and Privacy in Agentic Workflows. APM systems introduce a unique attack surface, requiring novel security models (mainly based on role-based access control). The capability of *Conversational Actionability* implies that agents may ingest malicious prompts (prompt injection) or interact with compromised external agents, potentially leading to data exfiltration or unauthorized actions. Simultaneously, *Self-Modification* capabilities introduce the risk of “poisoning” agents’ memories [78] to ultimately cause them to adopt unsafe behavioral patterns. Furthermore, while *explainability* is required for trust, it presents a privacy paradox: detailed explanations of agent reasoning may inadvertently leak sensitive business data or personally identifiable information to unauthorized third parties. To prevent deliberately malicious actors from modifying workflows, APM systems must adopt advanced defense mechanisms. Technical and semantic guardrails (such as gateways and strict input/output sanitization) can intercept malicious instructions, and more advanced architectural design patterns such as “Action-Selector” or “Plan-Then-Execute” can constrain agents to predefined API calls, preventing them from executing unauthorized workflow modifications [79]. Moreover, process mining techniques can be repurposed as real-time security monitors to automatically detect potentially malicious deviations in process execution [80]. However, genAI agent security is an emerging and evolving area of research (cf. [81] for a taxonomic overview) and, accordingly, APM systems face—just like any other systems that deploy genAI agents—many security challenges that remain unsolved. An example of an open challenge is “indirect prompt injection” [82], where malicious instructions are hidden within external data or documents the agent is tasked to process. Thus, a major cross-cutting challenge is the development of security frameworks that can govern the trade-off between agent autonomy and information security. As overly rigid guard-railing limits adaptive capabilities, it is important to find a balance that ensures agents can negotiate and adapt without compromising the integrity and confidentiality of the process-aware environment.

C3: Benchmarking and Evaluation Frameworks for APM. While individual capabilities of AI models (e.g., LLM reasoning) are frequently benchmarked, there is a distinct lack of holistic evaluation frameworks for agentic systems in a process management context. In addition to well-established BPM characteristics, such as time, cost, and quality, we require novel metrics to assess how well the agents in APM systems deliver their capabilities (see Section 3). While this also calls for novel benchmark datasets to evaluate how well agents adhere to these qualities in stochastic environments, the benchmark datasets may be compromised by *data pollution* (aka. *data contamination* or *benchmark contamination*) [83, 84]. This occurs when an AI model, used as a tool in the APM system, has been accidentally or intentionally exposed to the benchmark data during its training or fine-tuning phase. This has several negative consequences: (a) The AI model’s performance will be artificially inflated, giving a false sense of its

capabilities; (b) the purpose of a benchmark is to objectively measure performance on unseen data, yet contamination undermines this, making it difficult to reliably perform comparisons; (c) the AI agents employing such a tool that excels on a contaminated benchmark are likely to perform poorly in real-world applications where they encounter truly novel data. These are very important challenges to overcome to move APM from theoretical viability to industrial adoption, allowing organizations to quantify the return on investment (ROI) and risk profile of deploying autonomous workforce agents.

C4: Liability and Accountability. The shift from human-centric to agentic process execution creates a “responsibility gap” that cuts across legal and organizational dimensions. When the agents in an APM system operate within *framed autonomy*, determining liability for adverse outcomes becomes complex: is the error attributable to the software developer, the organization enforcing the regulations (frame) onto the APM system, the specific AI model provider, or the emergent behavior of the multi-agent system? This challenge is exacerbated by *self-modification*, where an agent’s behavior may drift significantly from its initial design. Research must align APM system architectures with emerging legal frameworks (such as the EU AI Act) to establish clear chains of accountability. This involves not only technical traceability but also the development of “social” contracts between human and digital agents, defining the boundaries of delegation and the precise conditions under which human intervention is legally and operationally mandatory.

C5: Methods for Engineering APM Systems. One complementary challenge is to build on the aforementioned challenges for establishing essential architectural and technical building blocks of agentic process management and deliver novel *analysis and design methods* to support the engineering of APM systems⁵. Such engineering methods may build on agent-oriented analysis and design methods, such as AAIL, Gaia, Tropos, or Prometheus [20].

5. Conclusions and Outlook

Our work extends the BPM community’s well-established tradition of writing manifestos. Notably, the *Process Mining Manifesto* [85] has stood the test of time as a rallying call for the community, facilitating the establishment of a data science tradition within BPM research and ultimately anticipating the emergence of process mining offerings as important components in large ERP vendors’ software suites. In the realm of BPM and AI, the community has developed several conceptual visions. Predating widespread LLM adoption and new-generation AI agents, the manifesto on *AI-augmented Business Process Management Systems* (ABPMS) focuses on the integration of AI capabilities into holistic process management and execution systems [23], introducing the vision of framing in BPM while leaving the agentic perspective *implicit*. As a complementary viewpoint, the *augmented process execution* proposal is

⁵e.g., see the CAiSE 2026 workshop EPAIS.

intelligence/analytics-oriented and provides a four-level pyramid, ranging from basic *descriptive* analytics via *prediction* and *prescription* to augmentation [86]. Paralleling the emergence of LLM-augmented process management software, the *Large Process Models* vision sketches and discusses AI-based BPM in the age of LLMs, highlighting LLM potential to facilitate some tasks, while also emphasizing that achieving self-improving processes (essentially in the sense of an ABPMS) requires, even in human-in-the-loop scenarios, more than current-state LLMs [59].

Overall, the following four vision statements have anticipated at least some ideas that are largely accepted in practice:

1. Intelligent business process execution requires the collaboration of autonomous software agents and humans;
2. These agents require symbolic *frames*;
3. While LLM-based process modeling and mining capabilities play a role in mainstream BPM software offerings, LLMs alone are insufficient for truly intelligent BPM systems.
4. More broadly, LLM-based technologies are driving the *autonomous enterprise* era through novel agentic AI platforms, making it crucial to examine their technological implications for agentic BPM platforms as a central backbone.

Continuing this tradition, we hope that our manifesto will both anticipate developments in BPM theory and practice and help researchers and practitioners embrace recent technological developments while planning for long-term sustainable impact on the discipline and the organizations that practice it. In this context, we highlight the interdisciplinary nature of APM research as a bridge between BPM, autonomous agents, and machine learning. This space is being significantly reshaped by the recent trend of LLM-based agents, often portrayed in industry as a silver bullet, an innovation wave that the BPM community should actively engage with.

Parts of the APM vision can be practically realized by applying recent advances in research areas such as declarative process specification and conversational process modeling and mining. Still, to fully realize APM systems, this manifesto should primarily be interpreted as a call to action for further research. Notably, we intentionally leave open the realization and implementation details of key functionalities such as process-awareness and framing, an agent’s framed knowledge and goals representation, as well as the aforementioned agent capabilities for explainability, conversational actionability, and self-modification. We invite diverse concrete implementations of the presented vision and conceptual architecture.

Future work could address challenges along (but not limited to) the following lines:

Establish formal abstractions for frames and goals. To better support framed autonomy in BPM, we suggest to (i) introduce first-class abstractions for mental models of processes, goals and normative frames; (ii) develop and evaluate algorithms for synthesizing provably frame-compliant and performant operational specifications from frames, goals, and environmental information; (iii) demonstrate the applicability of the abstractions and algorithms in the context of real-world business information systems.

Ensure explainability of *execution* and *management*. Explainability for APM systems requires capabilities of technically sound explanations on process, agent, and MAS levels that are dynamically adjusted such that they are interpretable for the potentially heterogeneous group of target agents. The explanations should be actionable on both instance (execution) and process (management) levels as a means to maximize agentic autonomy.

Ensure actionability of conversational approaches. Ensuring that agents behave faithfully and efficiently with respect to process goals requires research on how software agents can converse with human principals and other software agents in a process context, as well as with tools and traditional process-aware systems, how to assess agent behavior, and how to trade off autonomy and control.

Move self-adapting agents towards enterprise reality. The deployment of concurrently evolving, process-aware, and self-adaptive agents in real-world enterprises requires new research on the governance of self-adaptation and systematic frameworks for the comprehensive assessment of the consequences of self-adaptation.

In this context, we advocate for research that (i) integrates perspectives from both BPM and agents research; (ii) advances strong formal foundations that give principle-based guarantees, (iii) develops engineering results based on re-usable software artifacts and reproducible empirical evaluations grounded in systematic benchmarks; (iv) ultimately studies the value provided to businesses and society at large. Finally, we emphasize that a sustainable roadmap towards APM requires considering *agent* notions that encompass not only LLM-based “genAI agents” but also include human agents that remain crucial for organizational and societal success.

Acknowledgments

The article was derived from working group notes as published in the report of Dagstuhl Seminar 25192, “AUTOBIZ: Pushing the Boundaries of AI-Driven Process Execution and Adaptation”; these notes were further refined into several peer-reviewed workshop papers for the PMAI workshop at ECAI 2025 [16, 15, 17, 18], which form the basis of Sections 3 and 4 of this paper.

AC conducted this research while enrolled in the Italian National Doctorate in Artificial Intelligence run by Sapienza University of Rome (CUP B53C23003500006, Mission 4 NRRP Generic Research Scholarship, Component 1). TK was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

- [1] M. Wooldridge, N. R. Jennings, Intelligent agents: Theory and practice, The knowledge engineering review 10 (2) (1995) 115–152. doi:10.1017/S0269888900008122.
- [2] Y. Shen, X. Zhang, The impact of artificial intelligence on employment: the role of virtual agglomeration, Humanities and Social Sciences Communications 11 (1) (2024) 1–14. doi:10.1057/s41599-024-02647-9.